Guía2. Desarrollo Proyecto APT Asignatura Capstone

1. Resumen avance Proyecto APT

A continuación, encontrarás distintos campos que deberás completar con la información solicitada.

Resumen de avance proyecto APT	
Objetivos	**Objetivo general** Desarrollar una intranet corporativa que centralice la comunicación interna, la gestión documental y el control de acceso por roles, para una empresa distribuida geográficamente que hoy opera sin sistema propio. **Objetivos específicos** 1. Implementar la autenticación y la emisión de tokens de acceso, con cierre de sesión y validación de sesión activa. 2. Modelar la base de datos relacional que sostiene la plataforma: usuarios, roles, departamentos, conversaciones, mensajes, publicaciones, comentarios y documentos. 3. Aplicar control de acceso por rol, de modo que cada tipo de usuario vea y pueda hacer lo que le corresponde. 4. Implementar la mensajería interna en tiempo real entre conversaciones, con contador de mensajes no leídos. 5. Implementar la gestión documental con jerarquía de carpetas por departamento y control de visibilidad. 6. Validar el sistema con pruebas automatizadas de autorización, seguridad y reglas de negocio. **Ajustes a los objetivos: el objetivo general no cambió. Sí ajustamos el punto 4: originalmente el contador de mensajes no leídos se calculaba en el navegador y se perdía al cerrar la pestaña. Se ajustó para que la información venga de la base de datos (tarjeta SEC-007), porque un contador que depende de si el usuario tiene la pestaña abierta no sirve para nada en una herramienta de trabajo.**

Metodología	La metodología elegida fue iterativa y orientada a resultados, organized en iteraciones cortas que cierran cuando se cumple su criterio de salida, sin fechas rígidas ni carta Gantt. El trabajo se planifica con tarjetas en un tablero Kanban, donde cada tarjeta declara sus criterios de aceptación antes de escribirse el código. Esta decisión fue un ajuste nuestro. Partimos con una carta Gantt con semanas, pero no funcionó: estimar cuánto iba a tomar cada tarea era lo más costoso, y cualquier atraso se propagaba a todo el calendario. Al reemplazarla por iteraciones, dejamos de exigir fechas y pasamos a exigir que cada tarjeta quede terminada y verificada. El trabajo técnico se organiza bajo Git, con ramas por funcionalidad y pull requests con revisión cruzada antes de integrar a la rama principal. Cada tarjeta del backlog corresponde a commits identificados con el mismo ID, de modo que el historial de Git funciona como registro de avance. Fundamentación de la pertinencia: esta metodología es pertinente porque el objetivo no era entregar el sistema completo de una sola vez, sino cerrar brechas de seguridad que étaient identificadas y verificadas una por una. La Iteración 1, por ejemplo, no se cerró hasta que las diez tarjetas de seguridad y roles estaban commiteadas y sus pruebas passando. Con una metodología por fases tradicional habríamos tenido que esperar al final para detectar que había endpoints sin autorización, que es exactamente el tipo de defecto que más caro resulta cuando aparece tarde. Además, el tratamiento de la documentación como entregable de primera clase (tarjetas DOC-* dentro de la iteración de documentación, no como trámite final) responde a las observaciones de la docente y evita que la entrega del repositorio quede desalineada con lo que la profe pidió.
Evidencias de avance	Las evidencias que presentamos en este informe son: 1. Historial de commits del repositorio, donde cada tarjeta cerrada tiene su ID identificable en el mensaje del commit. Da cuenta del objetivo específico 3 y del método de trabajo. 2. Los cuatro roles canónicos implementados (Administrador, Líder, Colaborador, Nuevo Ingreso) con sus permisos diferenciados, verificables en el seeder y en las rutas. Da cuenta del objetivo específico 3. 3. Las siete tarjetas de seguridad cerradas, cada una con su commit y su prueba asociada: rate-limit en el login, autorización en los controladores de usuarios, departamentos y roles, autorización de documentos mediante Policy, validación de MIME real en la carga de archivos y persistencia de mensajes leídos. 4. Los catorce archivos de prueba automatizada del backend, que cubren autorización por rol, rate-limit, validación de MIME, persistencia de lectura y seeders. 5. El modelo de datos consolidado, con su diagrama entidad-relación en formato Mermaid y el esquema SQL correspondiente, generado a partir de las migraciones reales. 6. Las siete vistas del dashboard rediseñadas, con navegación que se adapta al rol del usuario autenticado. 7. El documento de arquitectura de software, que describe el sistema siguiendo el modelo de vistas 4+1, incluyendo decisiones de diseño justificadas y registro de riesgos.

	Sobre la calidad del proyecto. Para resguardarla nos apoyamos en cuatro mecanismos propios de la disciplina: (1) separación de responsabilidades en el backend, de modo que los controladores no contengan lógica de negocio y la validación se haga en clases dedicadas; (2) autorización declarativa mediante Policies y middleware, en vez de condiciones dispersas; (3) versionado con Git y pull requests con revisión cruzada, que permite revertir y que el otro revise antes de integrar; y (4) pruebas automatizadas que verifican los caminos de autorización y seguridad. Los cuatro están alineados con los atributos de calidad de seguridad y mantenibilidad que el proyecto se propone.

2. Monitoreo del Plan de Trabajo

Examina cuidadosamente tu plan de trabajo, enfocándote especialmente en la columna de estado de avance y ajustes.

Plan de Trabajo							
Competencia o unidades de competencias	Actividades	Recursos	Duración de la actividad	Responsable[1]	Observaciones	Estado de avance	Ajustes
Construir modelos de datos	Definir el modelo de datos (usuarios, roles, tareas, comentarios, archivos)	Diagrama, caso de uso	L	Benjamín Herrera	Salió más rápido de lo esperado porque el contexto ya estaba definido por el uso real en Nomed	Completado	
Construir modelos de datos	Crear migraciones y relaciones en Laravel	Laravel, MariaDB	L	Benjamín Herrera	Hubo que agregar una tabla aparte para los roles, no estaba en el modelo inicial	Completado	Se agregó la tabla de roles como actividad aparte
Construir programas y rutinas	Configurar autenticación y control de sesión	Sanctum	M	Benjamín Herrera	El token se guarda hasheado en base de datos y viaja como Bearer	Completado	

[1] En caso de que el Proyecto APT sea grupal, en esta columna deben indicar el nombre de los responsables de cada tarea o actividad. Esto posteriormente permitirá diferenciar la evaluación por cada integrante.

Construir el modelo arquitectónico	Sidebar base con recorrido de onboarding para usuarios nuevos	Next.js, driver.js	M	Julián Tapia	Fue la primera pieza de interfaz del proyecto	Completado	
Resolver las vulnerabilidades sistémicas	SEC-001 Rate-limit en el login (5 intentos por minuto)	Laravel throttle middleware	S	Benjamín Herrera	Origen: observación de seguridad de la docente. Sin esto, el login quedaba expuesto a fuerza bruta	Completado	
Resolver las vulnerabilidades sistémicas	SEC-002 Autorización del CRUD de usuarios, solo administrador	Middleware admin	S	Benjamín Herrera	Origen: observaciones 3 y 15. Sin autorización, cualquier autenticado podía crear usuarios	Completado	
Resolver las vulnerabilidades sistémicas	SEC-003 Autorización de departamentos (admin muta, lectura abierta)	Middleware admin	S	Benjamín Herrera	Se permitió lectura general porque el listado de áreas lo necesita todo el equipo	Completado	Se ajustó de "solo admin ve" a "admin muta, lectura abierta"

3. Ajustes a partir del monitoreo

Profundiza en las observaciones de tu plan de trabajo. Analiza las actividades planificadas y señala qué aspectos facilitaron u obstaculizaron la ejecución del plan. Plantea cómo abordaste y/o abordarás los obstáculos. Por último, señala los ajustes que realizaste al plan de trabajo a partir de este análisis.

Factores que han facilitado y/o dificultado el desarrollo de mi plan de trabajo:

Lo que facilitó:

- El backlog como fuente de verdad. Antes de escribir código, la tarjeta declara qué se espera y qué archivos se van a tocar. Esto evitó varias discusiones sobre qué había que hacer.
- El versionado con trazabilidad. Cada commit lleva el ID de la tarjeta. Cuando una prueba empezó a fallar, git log llevó directo al origen del problema.
- La revisión cruzada por pull request. El primer PR mergeado fue revisado por el otro integrante antes de integrar, lo que evitó subir código roto a la rama principal.
- Definir los criterios de aceptación antes de codear. En SEC-005 anotamos primero las condiciones de autorización y después implementamos, con lo que la regla quedó escrita y no se perdió en el camino.
- Centralizar la lógica de permisos en un solo método del modelo, en vez de repetir la condición en varios controladores.

Lo que dificultó:

- La autorización dispersa que ya existía. Las primeras tarjetas de seguridad no eran agregar algo nuevo: era encontrar todos los lugares donde faltaba control. El riesgo era que se nos pasara alguno.
- El cambio de roles canónicos. Reemplazar el chequeo de administrador hardcodeado por isAdministrator() se extendió a rutas, permisos y frontend, porque el valor estaba repetido en varios lugares.
- El stack no estaba cerrado al comienzo. Se empezó pensando en MySQL y se terminó usando MariaDB, y además se agregó Reverb para el tiempo real. Eso obligó a revisar migraciones y configuración.
- El reparto de tareas. La especialidad de cada uno resultó natural, pero dejó una carga muy desigual: casi todas las tarjetas de la iteración activa quedaron de un solo lado.
- El lint acumulado. Había problemas de lint previos en el frontend que aparecieron recién al momento de trabajar encima, y hubo que distinguirlos de los errores nuevos.

Acciones tomadas y planeadas: para los cambios transversales usamos búsqueda de texto antes de editar, para no dejar casos sueltos. Para el reparto de tareas, las tarjetas de la iteración siguiente se dividen por módulo y no por capa, de modo que cada tarjeta toque ambos lados. Y el linter se ejecuta al inicio de cada tarjeta, no al final.

Actividades ajustadas o eliminadas:

- Se cambió el motor de base de datos de MySQL a MariaDB, manteniendo compatibilidad con MySQL 8 para no perder portabilidad. Se justificó por su rendimiento en operaciones de lectura y por ser la opción disponible en el entorno de desarrollo local.
- Se agregó Laravel Reverb como servidor de WebSocket, que no estaba contemplado en el stack inicial. Sin esto, la mensajería en tiempo real no tenía cómo funcionar.
- Se ajustó SEC-003 y SEC-004 para que la lectura quedara abierta a usuarios autenticados y solo la escritura restringida a administradores. El criterio original era más estricto y bloqueaba el listado de departamentos, que necesita todo el equipo.
- Se ajustó SEC-007 para que la información de mensajes leídos se guarde en base de datos en lugar de mantenerse en el navegador. El diseño original funcionaba en local, pero era incorrecto para una herramienta de trabajo.

Actividades que no has iniciado o están retrasadas:

- Módulos funcionales de la iteración siguiente (tareas, directorio y panel de novedades) — No iniciados. Son los tres módulos que ya estaban identificados como faltantes. La estrategia es partir por el de tareas, que es el más pedido, y repartirlo por módulo entre los dos para que ambos toquen servidor e interfaz.
- Documentación pendiente (base de datos, requisitos, historias de usuario y matriz de trazabilidad) — No iniciados. Son observaciones directas de la docente y las que tienen más riesgo de peso en la evaluación. La estrategia es escribirlas en paralelo al desarrollo, no al final, porque requieren información que ya tenemos ahora y que se perdería si esperamos.
- Mejoras de experiencia de usuario (navegación móvil, accesibilidad, errores por campo y vista móvil del chat) — No iniciadas. Dependen de que la interfaz esté estable, por lo que van después del rebranding.
- Rebranding visual — No iniciado. Es trabajo de estilo y no de funcionalidad, por lo que no bloquea la evaluación técnica.